

Capstone Project

Proposals due: before Friday, July 31, 2026 (last accepted 11:59 PM Thursday, July 30)

Build period: the final 3 weeks of the program

Presentations: the last 2 class days

Teams: already assigned

Overview

The capstone is the largest project of the program. Over the final three weeks, your team will design, build, and present one substantial project, chosen from the proposals you submit. It is your chance to show what you can do, to build something you are proud to put in front of an employer, and to practice the full arc of real work: proposing, scoping, building, and communicating.

Every capstone falls into **one of three domains**:

- **Full Stack Development:** an end to end application, with a frontend client, a backend server, and a database.
- **Data Science:** exploring the correlation between multiple datasets to surface insights on how to improve life for New York City residents.
- **Cybersecurity:** a finished **threat intelligence assessment** of a real Advanced Persistent Threat group, judging how likely it is to threaten a given sector in the next six months.

Each domain has its own detailed requirements below. The **shared requirements** and the **proposal process** apply to every project regardless of domain.

Start with a problem, not a technology. The best projects answer a real question or solve a real annoyance. Pick something you actually care about, because you will be living with it for three weeks.

How Proposals Work

Your team submits **three project proposals** before **Friday, July 31, 2026**. The last accepted time is **11:59 PM on Thursday, July 30**. They may be:

- all in the **same domain**,
- or a **mix** of domains.

The **instructional staff will review all three and select one** for your team to build. We reserve the right to **modify a proposal**, or to **reject all three** and steer your team in a different direction, when we believe it will lead to a better learning experience.

If your team does not submit proposals before the deadline, or does not submit any viable proposals, the instructional team will **assign a project to your team**.

Each proposal must include:

1. **Domain** (full stack, data science, or cybersecurity)
2. **Purpose or research question**: what you want to build, or the question you want to explore
3. **Tech stack**: the main tools, frameworks, datasets, or sources you plan to use
4. **Guaranteed scope**: the features or results you are confident you can deliver in three weeks (your minimum)
5. **Tiered learning goals**: things you may be able to reach if time allows
6. **How the work will be divided**: roughly how your team will split the labor. You do not have to put one person on one feature; teammates can work on the same piece together (for example, pair programming)

We are looking for a clear idea, not a finished design.

Submit early if you can. If we spot something that needs a tweak, we will tell you before the deadline so you can revise and resubmit.

How to submit: post your three proposals through the **Google Form** linked in Slack. This is a **separate** form from the project repo submission you have used for past projects.

Timeline

When	What
before Friday, July 31, 2026	Submit 3 proposals (via the Google Form in Slack)
Start of the capstone	Staff selects a proposal, scope confirmed with an instructor
Weeks 1 to 2	Build the minimum version end to end, then iterate
Week 3	Polish, test, write your README, prepare your presentation
Last 2 class days	Presentations

Aim to have a **working minimum version early**, then improve it. A small project that works beats a large one that does not.

Domain: Full Stack Development

Build a working **application** that a real person could use, with a frontend, a backend, and data that persists.

Required:

- A **frontend** built with **Ionic React** (from Week 5).
- A **backend** you wrote: an **Express** server exposing an API.
- **Persistent data** in **PostgreSQL** with **Prisma** (from Week 6). Data stored on the **server** must persist across restarts.
- A **data model with relationships** (more than one table that reference each other).
- **Full CRUD** on at least one core resource, through your own API **routes**: a create, read, update, and delete **endpoint**.
- **Polling**: the client periodically re-fetches from the server so the UI reflects **shared or changing state** without a manual refresh. (Plan for an app that has state worth refreshing.)
- **Input validation and security**: validate and sanitize everything coming from the client, and never trust it. The instructional team reserves the right to try to **break or hack** your project.

Shared requirements (all domains):

- Keep all work in a **GitHub repo** with **meaningful commits from every member**. We look at the commit history.
- Write a **README** that explains what the project is, how to run or reproduce it, and who did what.
- **Keep secrets and large files out of git**: use a `.gitignore`, and never commit API keys, passwords, or datasets over a few megabytes. Provide setup or download instructions instead.
- You may use **AI tools** at any stage, but you must **disclose** how and where you used them (a short section in your README is fine). Using AI well is a skill; hiding it is not.
- Be ready to **explain any part of your project**. You should understand every line you ship, whoever or whatever wrote it first.

Tiered Learning Goals:

- Persist data on the **client** too (for example local storage or an on-device database), so the app keeps state without the server.
- **Security hardening**: rate limiting, parameterized queries, protection against XSS and CSRF, and no secrets in the repo.
- Upgrade polling to **real-time updates** with **WebSockets**.
- **Automated tests** for your core logic (unit and integration).
- Integrate a **third-party API**.

Domain: Data Science

Explore multiple datasets to understand what they can, and cannot, tell you. Practice the principles of **exploratory data analysis (EDA)**: start from a question, clean and probe the data, visualize it, and let what you find shape the next question. An answer or a recommendation may emerge along the way, but the goal is honest, rigorous exploration, not forcing a conclusion.

Required:

- **Start with up to 3 focused, specific questions** to guide your exploration, chosen before you settle on the data. More questions will surface as you dig in, which is expected and welcome. Good questions are **specific and unbiased**:
 - *Which subway lines had the most weekend delays in 2024?*

- *Do neighborhoods with more tree canopy report fewer heat complaints?*
- Avoid questions that assume their own answer, like *Why is the L the worst line?*
- **At least three datasets**, with **at least one** from a public source such as [NYC Open Data](#). The datasets must genuinely **work together** (merged on a shared key, compared, or one giving context for another).
- **Data cleaning and wrangling**: handle types, missing values, duplicates, and outliers, and show your work.
- **Exploratory analysis** with **static visualizations** (**seaborn** or **matplotlib**) that support your points.
- An **interactive dashboard** of your findings, built with **Recharts** in an **Ionic** app, with **interactive filters** (for example date range, category, or borough).
- A **report for a decision-maker**, delivered as a **PDF or slideshow** and separate from your notebook: what you found, any recommendation the data supports, and **what the data does not show**. Write it for a **CEO**, not a data scientist. Be careful with **correlation versus causation**.
- A **reproducible notebook** that reads like a report, with **presentational markdown cells** explaining your reasoning around the code. Put **dataset download instructions in the first cell**.

Shared requirements (all domains):

- Keep all work in a **GitHub repo** with **meaningful commits from every member**. We look at the commit history.
- Write a **README** that explains what the project is, how to run or reproduce it, and who did what.
- **Keep secrets and large files out of git**: use a `.gitignore`, and never commit API keys, passwords, or datasets over a few megabytes. Provide setup or download instructions instead.
- You may use **AI tools** at any stage, but you must **disclose** how and where you used them (a short section in your README is fine). Using AI well is a skill; hiding it is not.
- Be ready to **explain any part of your project**. You should understand every line you ship, whoever or whatever wrote it first.

Tiered Learning Goals:

- Fit a **machine learning model** (linear regression, k-means, and so on with **scikit-learn**) if a prediction question emerges from your exploration, evaluated **honestly** with a **train and test split**. Report the test score, not the training score.

- **Statistical rigor:** back a claim with a confidence interval, a hypothesis test, or by controlling for a confounder.
- **Deploy** your dashboard so anyone can use it.
- Bring in **even more datasets**, or a much larger one, for richer context.

What good looks like: an honest exploration that shows its cleaning, follows the data where it leads, and knows the limits of its claims. A modest finding, well defended, beats a dramatic one that the data cannot support.

Domain: Cybersecurity

Produce a finished **threat intelligence assessment** of a real **Advanced Persistent Threat (APT)** group. Pick an active or recently active APT, research it across multiple source types, and answer one question:

How likely is this APT to pose a direct threat to a specified sector or organization type within the next six months?

This is a **research, analysis, and briefing** project, not a coding project. You will apply the full **intelligence cycle** practiced during the cybersecurity block.

Required:

- **Source and grade at least five independent sources** on your chosen APT, spanning **at least three collection disciplines** (for example: a government advisory, vendor incident-response reporting, a single-actor deep report, financial or blockchain tracing, or OSINT).
- **Map the APT's documented TTPs** to the [MITRE ATT&CK](#) framework.
- Run an **Analysis of Competing Hypotheses (ACH)** on a scoped question about the APT's **intent, capability, or attribution**.
- Write a **finished intelligence product** using **BLUF** structure, consistent **estimative language**, an **actor profile**, and at least one **Diamond Model** summary of a documented intrusion.
- Deliver a **verbal briefing** to a mock stakeholder, stating a **confidence level** and the reasoning behind it.

Shared requirements (all domains):

- Keep all work in a **GitHub repo** with **meaningful commits from every member**. We look at the commit history.
- Write a **README** that explains what the project is, how to run or reproduce it, and who did what.
- **Keep secrets and large files out of git**: use a `.gitignore`, and never commit API keys, passwords, or datasets over a few megabytes. Provide setup or download instructions instead.
- You may use **AI tools** at any stage, but you must **disclose** how and where you used them (a short section in your README is fine). Using AI well is a skill; hiding it is not.
- Be ready to **explain any part of your project**. You should understand every line you ship, whoever or whatever wrote it first.

Deliverables:

1. A **source log** with reliability and credibility grades for each source used.
2. A completed **ACH matrix**: the scoped question, the hypotheses, and the evidence mapping.
3. An **ATT&CK mapping** of the APT's known kill chain.
4. A **written assessment** (2 to 4 pages, estimative language throughout).
5. A **stakeholder briefing**, taking at least one follow-up question from the panel.

Confidence level standard: the assessment must state a confidence level on the estimative scale from production day (**very high, high, medium, low, very low**), and must **justify that level** by pointing to specific evidence in the ACH matrix and source grading, not a general impression.

Responsible research: work only from **open, published sources**. Do not attempt to contact, engage with, or interact with threat actors or their infrastructure.

Tiered Learning Goals:

- Add a second **structured analytic technique** (for example a key assumptions check) alongside the ACH.
- Turn your ATT&CK mapping into detection or mitigation **recommendations for a defender** in the target sector.
- Build a clear **visual**: an intrusion timeline, or an ATT&CK Navigator layer for the actor.

What good looks like: a well-sourced, clearly-reasoned assessment whose confidence level is **earned by the evidence**, and a briefing a non-specialist stakeholder could act on.

Deliverables (all domains)

1. A **GitHub repository** with your code, notebook, or written deliverables, a clear **README**, and commit history from every member.
2. A **presentation** (about 15 minutes; for cybersecurity, this is your **stakeholder briefing**) covering:
 - The **problem** and why it matters
 - Your **approach** and the key decisions you made
 - A **demo** or walkthrough of the working result
 - What was **hard**, and what you would do next
 - **Where and how you used AI**

What We Are Looking For

- A **real problem**, clearly stated, and a result that honestly addresses it.
- **Working software or a reproducible analysis**, not a slide deck of intentions.
- **Sound fundamentals** for the domain: data modeling and auth for full stack, honest evaluation for data science, rigorous sourcing and earned confidence levels for security.
- **Teamwork** visible in the commit history.
- An **honest account** of the limits: what does not work yet, what the data cannot prove, what you would harden next.
- A **clear presentation** that a non expert could follow.

An Example Capstone

For a sense of the scope we are aiming for, look at **philosoph**, a project your instructor built. It is a classroom quiz game, and it touches several skillsets from this course across more than one domain:

- **Full stack development**: an **Ionic React** frontend and an **Express** plus **TypeScript** backend, with **WebSockets** for the live game and **TanStack Query** on the client.
- **Analytics and reporting**: **Recharts** dashboards for the educator, session data exported to **CSV**, and a command-line **report generator** that produces per-student and class-wide **PDF** summaries.

- **Ethical AI use:** every prompt behind it is recorded in `COLLABORATION.md`, the same kind of AI disclosure we ask of you.

You do not need to match its size. It is here to show what one focused, working project can look like end to end.

github.com/jonathan-chin/philosoph

Final Submission

This is the **end of the capstone**, separate from the proposal step above.

- Push your final work to **GitHub**.
- Submit your **repo link** and your **presentation** via the **project repo form** in Slack (the same one you have used for past projects, not the proposal form).
- **Present in class** on your assigned day.